Dimensionality Reduction — Curse of Dimensionality, PCA, LDA, t-SNE

> Professional briefing: forward-looking, actionable, and ready for integration into ML pipelines. This document contains in-depth conceptual notes, mathematics where essential, practical guidance, and a full Jupyter notebook code block that implements PCA, LDA, and t-SNE on the Iris dataset (plus classification comparisons).

Executive summary

- **Curse of Dimensionality**: A set of phenomena that make high-dimensional data hard to analyze — sparsity, distance concentration, exponential growth of volume, and increased sample complexity. Mitigation: feature selection, dimensionality reduction, regularization, and models that exploit sparsity.

- **PCA (Principal Component Analysis)**: Linear, unsupervised method that finds orthogonal directions of maximal variance. Use for denoising, compression, visualization; baseline first step.

- **LDA (Linear Discriminant Analysis)**: Supervised linear method that finds projections maximizing class separability (ratio of between-class to within-class scatter). Useful when labels exist and class-conditional covariances are similar.

- **t-SNE (t-distributed Stochastic Neighbor Embedding)**: Nonlinear stochastic visualization method that preserves local neighborhoods; great for exploratory plots but not for downstream models. Prefer UMAP in modern workflows for speed and transformability.

- **Practical posture**: Start with PCA for baseline; use LDA for supervised low-class-count problems; use UMAP for visualization by default, t-SNE when you need its specific aesthetics. For production embeddings, prefer learned embeddings (autoencoders, contrastive learning) or deterministic transforms.

---

1) Curse of Dimensionality — detailed

Definition & intuition - As feature dimension d increases, the sample size required to densely sample the space grows exponentially. Points become sparse and pairwise distances concentrate.

Concrete consequences - Distance metrics lose meaning (distance concentration). - Nonparametric methods (k-NN, density estimation) degrade. - Optimization and model variance increase; overfitting risk rises.

Mitigation strategies - Dimensionality reduction (PCA, autoencoders, random projection). - Feature selection (statistical tests, mutual information, embedded methods like LASSO). - Regularization and model selection.

Rule of thumb - If `d > 50` and `n < 10k`, explicitly consider dimensionality reduction.

---

2) Principal Component Analysis (PCA)

Goal: Find orthogonal axes (principal components) that capture maximal variance. Projection minimizes mean squared reconstruction error for chosen dimension `k`.

Math (compact) - Center data `X` (zero mean). - Covariance: `C = (1/(n-1)) X^T X`. - Eigen-decomposition: `C v = lambda v`. - Sort eigenvectors `v` by eigenvalues `lambda`; first `k` form projection matrix `W_k`. - Projection: `Z = X W_k`.

Properties - Linear. Deterministic. Optimal for MSE reconstruction. - Sensitive to feature scaling.

When to use - Noise reduction, compression, exploratory visualization, speeding up pipelines, initializing algorithms.

Alternatives - Kernel PCA (nonlinear), Random Projection (fast approx), Autoencoders (learned nonlinear), ICA (statistical independence).

Practical tips - Standardize features (zero mean, unit variance) unless units are meaningful. - Use randomized SVD for large problems. - Choose `k` by explained variance or downstream validation.

---

3) Linear Discriminant Analysis (LDA)

Goal: Find linear projections that maximize class separability.

Math (compact) - Within-class scatter `S_W = sum_c sum_{x in c} (x - mu_c)(x - mu_c)^T`. - Between-class scatter `S_B = sum_c n_c (mu_c - mu)(mu_c - mu)^T`. - Solve generalized eigenproblem: `S_W^{-1} S_B v = lambda v`. - At most `C-1` components (C = number of classes).

Assumptions - Class-conditional Gaussians with equal covariance matrices.

When to prefer - Supervised compression aiming at classification. Small number of classes. When model interpretability and class separation matter.

Alternatives - QDA (relaxes equal covariance), logistic regression, SVM, metric learning (triplet/contrastive) for nonlinear discriminative embeddings.

Practical pattern - If `d >> n`, do PCA first to stabilize covariance estimation (PCA -> LDA = Fisherfaces approach).

---

4) t-SNE

Goal: Nonlinear visualization preserving local neighborhoods via probability distributions and KL divergence.

Key mechanics - Convert pairwise distances in high-d to conditional probabilities (Gaussian kernel with perplexity controlling neighborhood size). - In low-d, model pairwise similarities with Student-T (heavy tail) to alleviate crowding. - Minimize KL divergence between high-d and low-d similarity distributions with gradient descent.

Hyperparameters - Perplexity (typical 5–50), learning rate, number of iterations, initialization (PCA recommended).

Limitations - Non-deterministic; not preserving global geometry; expensive for large `n`; not for downstream tasks as-is.

Alternatives - UMAP: faster, preserves more global structure, supports transform on new data; generally the modern default.

---

5) Practical recipes & decision matrix

1. **EDA/visualization**: scale → PCA (30–50 dims) → UMAP/t-SNE to 2D.
2. **Preprocessing for ML**: linear suspected → PCA; supervised → LDA; complex nonlinear → autoencoder/contrastive.
3. **Small-sample, high-d**: PCA→LDA; feature selection.
4. **Production**: choose deterministic transforms you can save and apply (PCA, trained autoencoder, saved UMAP transformer).

---

6) Jupyter notebook code (Iris dataset)

Below is a ready-to-run Jupyter notebook code block. The same notebook has been executed and saved as a downloadable `.ipynb` in the workspace; run it locally or in your environment.

```
# Dimensionality_Reduction_Primer.py
# Notebook: Demonstrates PCA, LDA, t-SNE on Iris dataset and compares
classification accuracy.

import numpy as np
```

```
import pandas as pd
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.manifold import TSNE
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.random_projection import GaussianRandomProjection
import matplotlib.pyplot as plt

# Load Iris
iris = datasets.load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names

# Scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# PCA
pca = PCA(n_components=2)
X_pca2 = pca.fit_transform(X_scaled)
explained = pca.explained_variance_ratio_

# LDA
lda = LinearDiscriminantAnalysis(n_components=2)
X_lda2 = lda.fit_transform(X_scaled, y)

# t-SNE (on PCA-reduced 4 dims for speed/stability)
pca4 = PCA(n_components=4)
X_pca4 = pca4.fit_transform(X_scaled)
tsne = TSNE(n_components=2, perplexity=30, init='pca', random_state=42)
X_tsne2 = tsne.fit_transform(X_pca4)

# Random Projection (fast baseline)
rp = GaussianRandomProjection(n_components=2, random_state=42)
X_rp2 = rp.fit_transform(X_scaled)

# Classification comparison (Logistic Regression)
clf = LogisticRegression(max_iter=200)
raw_score = cross_val_score(clf, X_scaled, y, cv=5).mean()
pca_score = cross_val_score(clf, X_pca2, y, cv=5).mean()
lda_score = cross_val_score(clf, X_lda2, y, cv=5).mean()
rp_score = cross_val_score(clf, X_rp2, y, cv=5).mean()

print('Logistic CV accuracy (raw):', raw_score)
```

```
print('Logistic CV accuracy (PCA2):', pca_score)
print('Logistic CV accuracy (LDA2):', lda_score)
print('Logistic CV accuracy (RP2):', rp_score)

# Plots
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].scatter(X_pca2[:,0], X_pca2[:,1], c=y)
axes[0].set_title('PCA (2D) - explained var: {:.2f}, {:.
2f}'.format(explained[0], explained[1]))
axes[1].scatter(X_lda2[:,0], X_lda2[:,1], c=y)
axes[1].set_title('LDA (2D)')
axes[2].scatter(X_tsne2[:,0], X_tsne2[:,1], c=y)
axes[2].set_title('t-SNE (2D, perplexity=30)')
plt.tight_layout()
plt.show()
```

Appendix: Quick decision checklist

- Start with PCA for baseline.
- Use LDA when you have labels and few classes; perform PCA first if `d >> n`.
- For visualization use UMAP (general default) or t-SNE when you need its particular clustering aesthetic.
- For production embeddings, prefer deterministic, transformable methods or trainable models whose weights you can version.

_End of document. If you want, I can export this as a PDF or split into slides for a presentation. The executed notebook and raw `.ipynb` file are also available for download (generated alongside this document)."}